



UNIVERSIDAD DE MÁLAGA

INFORMÁTICA INDUSTRIAL

Memoria Técnica: Sistema IoT para Invernadero Automatizado

Grupo 11

Autores:

Arturo Rios Pérez

Rafael Martín Benítez

Adrián Sola Nieto

11 de enero de 2026

Índice

1. Introducción y Objetivos	2
1.1. Descripción del Proyecto	2
1.2. Objetivos	2
1.3. Alcance y Funcionalidades	3
2. Diseño Hardware y Esquema de Conexionado	4
2.1. Nodo de Lectura (Sensores)	4
2.1.1. Componentes y Conexionado	4
2.2. Nodo de Actuadores	4
2.2.1. Componentes y Conexionado	5
2.3. Nodo de Cámara (Vigilancia)	5
2.3.1. Configuración Hardware	5
3. Diseño Software	7
3.1. Firmware de los Dispositivos IoT	7
3.1.1. Nodo de Sensores: Multitarea con FreeRTOS	7
3.1.2. Nodo de Actuadores: Suscripción y Reacción	7
3.1.3. Nodo de Cámara: Captura Híbrida (MQTT/Temporizador) y Envío HTTP	8
3.2. Aplicación de Control y Supervisión (Node-RED)	9
3.2.1. Gestión de Datos y Base de Datos	9
3.2.2. Lógica de Automatización	9
3.2.3. Integración con Servicios Externos (AEMET)	9
3.3. Interfaces de Usuario	10
3.3.1. Dashboard Web	10
3.3.2. Bot de Telegram	11
4. Técnicas de Robustez y Eficiencia	12
4.1. Gestión de Energía y Estabilidad	12
4.2. Tolerancia a Fallos de Comunicación	12
4.3. Multitarea (FreeRTOS)	12
5. Librerías Utilizadas	13
6. Resultados y Conclusiones	13
6.1. Logros Alcanzados	13
6.2. Conclusión	14
7. Lista de Ficheros Entregados	14

1. Introducción y Objetivos

1.1. Descripción del Proyecto

El presente documento detalla el diseño, desarrollo e implementación de una solución completa en el ámbito del “Internet de las Cosas” (IoT), aplicada a la automatización y monitorización de un invernadero inteligente. Este proyecto surge como respuesta a los requisitos planteados en la asignatura de Informática Industrial, buscando integrar tecnologías de hardware embebido, protocolos de comunicación en red y supervisión remota.

La solución propuesta se basa en una arquitectura distribuida compuesta por múltiples nodos de procesamiento basados en el microcontrolador ESP32. El sistema es capaz de monitorizar variables ambientales críticas (temperatura, humedad del aire, humedad del suelo y luminosidad) y actuar sobre el entorno físico mediante sistemas de riego y ventilación. Además, se ha integrado un sistema de vigilancia visual mediante captura de imágenes y una interfaz de usuario avanzada basada en Node-RED, accesible tanto vía web (Dashboard) como mediante mensajería instantánea (Telegram).

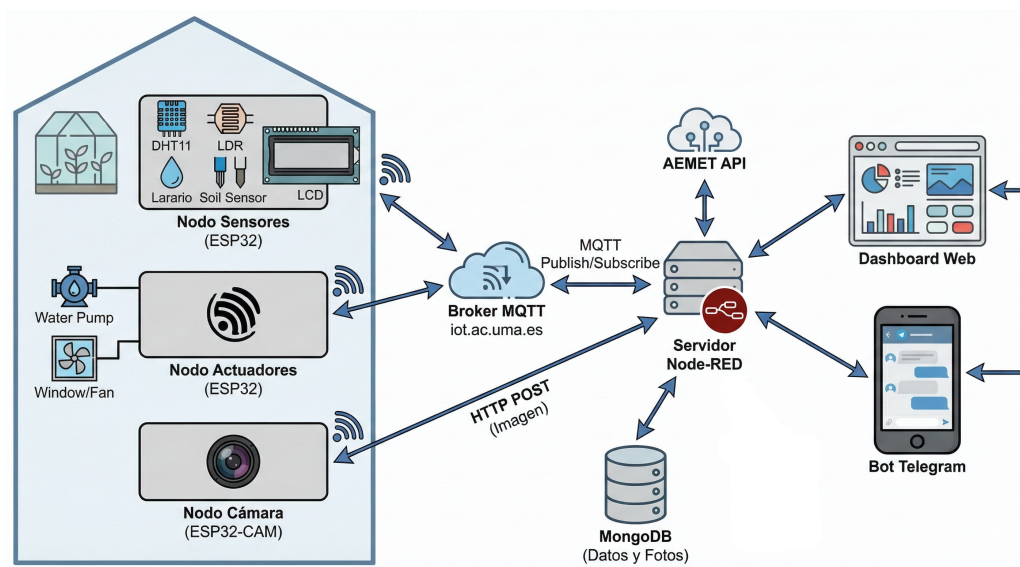


Figura 1: Diagrama conceptual de la arquitectura del sistema Invernadero IoT.

1.2. Objetivos

El objetivo principal del proyecto es crear un sistema robusto, escalable y amigable para el usuario que permita la gestión integral de un invernadero sin necesidad de presencia física constante. Para ello, se han establecido los siguientes objetivos específicos funcionales y técnicos:

- **Adquisición de Datos:** Lectura periódica y precisa de sensores ambientales utilizando un nodo dedicado basado en ESP32, implementando estrategias de multitarea con FreeRTOS para asegurar la eficiencia temporal.

- **Actuación Remota y Automática:** Control de dispositivos físicos, incluyendo una bomba de agua para el riego y un servomotor junto a ventiladores para la aireación del recinto. Este control debe ser tanto manual (bajo demanda del usuario) como automático (basado en umbrales de sensores).
- **Supervisión Visual:** Implementación de un nodo de cámara (ESP32-CAM) que capture imágenes del estado del invernadero y las envíe a un servidor central para su almacenamiento y visualización, optimizando el consumo energético mediante modos de bajo consumo (*Deep Sleep*).
- **Comunicaciones IoT:** Despliegue de una red de comunicaciones basada en el protocolo MQTT para el intercambio de mensajes entre los nodos y el servidor central, garantizando una topología desacoplada y escalable.
- **Interfaz de Usuario y Almacenamiento:** Desarrollo de un sistema en la nube utilizando Node-RED para la orquestación de mensajes, un Dashboard web para la visualización gráfica de históricos y control en tiempo real, y un Bot de Telegram para notificaciones y comandos conversacionales. Además, incluye almacenamiento persistente de datos en base de datos MongoDB.

1.3. Alcance y Funcionalidades

El sistema desarrollado cumple con los requerimientos funcionales descritos en la propuesta del proyecto, abarcando:

- Lectura de temperatura y humedad ambiental (DHT11), luminosidad (LDR) y humedad del suelo.
- Visualización local de datos mediante pantalla LCD I2C en el nodo de sensores.
- Comunicación inalámbrica vía WiFi con un broker MQTT centralizado (`iot.ac.uma.es`).
- Lógica de control distribuida y centralizada mediante flujos de Node-RED.
- Integración de servicios externos, como la consulta de predicción meteorológica a la AE-MET para complementar la toma de decisiones.

2. Diseño Hardware y Esquema de Conexionado

El sistema se ha dividido en tres nodos independientes para modularizar las funciones de lectura, actuación y vigilancia. Para ello, se han utilizado microcontroladores ESP32 debido a su conectividad WiFi y capacidad de procesamiento. A continuación, se detalla el hardware y el conexionado de cada nodo.

2.1. Nodo de Lectura (Sensores)

Este nodo es el encargado de monitorizar las condiciones ambientales del invernadero. Está basado en una placa de desarrollo ESP32 e integra sensores para medir temperatura, humedad relativa, humedad del suelo y luminosidad.

2.1.1. Componentes y Conexionado

Se han utilizado los siguientes pines GPIO del ESP32, definidos en el firmware de lectura:

- **Sensor DHT11 (Temperatura y Humedad Aire):** Conectado al pin **GPIO 33**. Este sensor digital proporciona lecturas de la temperatura ambiente y la humedad relativa dentro del invernadero.
- **Fotorresistencia LDR (Luminosidad):** Conectada al pin analógico **GPIO 34**. Se ha configurado con una resolución de lectura de 12 bits (valores de 0 a 4095) para medir la intensidad lumínica incidente.
- **Sensor de Humedad de Suelo (Capacitivo/Resistivo):** Conectado al pin analógico **GPIO 35**. Permite determinar la necesidad de riego midiendo la conductividad del sustrato.
- **Pantalla LCD (16x2) con adaptador I2C:** Utilizada para mostrar localmente los valores leídos. Se conecta a través del bus I2C estándar del ESP32:
 - SDA: **GPIO 21**
 - SCL: **GPIO 22**

El dispositivo se alimenta a través del puerto USB (5V), distribuyendo la alimentación a los sensores que lo requieren (VCC y GND). Además, posee de un interruptor analógico para apagar la pantalla y mejorar la eficiencia energética del invernadero.

2.2. Nodo de Actuadores

Este nodo gestiona los elementos activos del invernadero encargados de modificar las condiciones ambientales (riego y ventilación). Al igual que el nodo de sensores, utiliza un ESP32 que recibe órdenes vía MQTT.

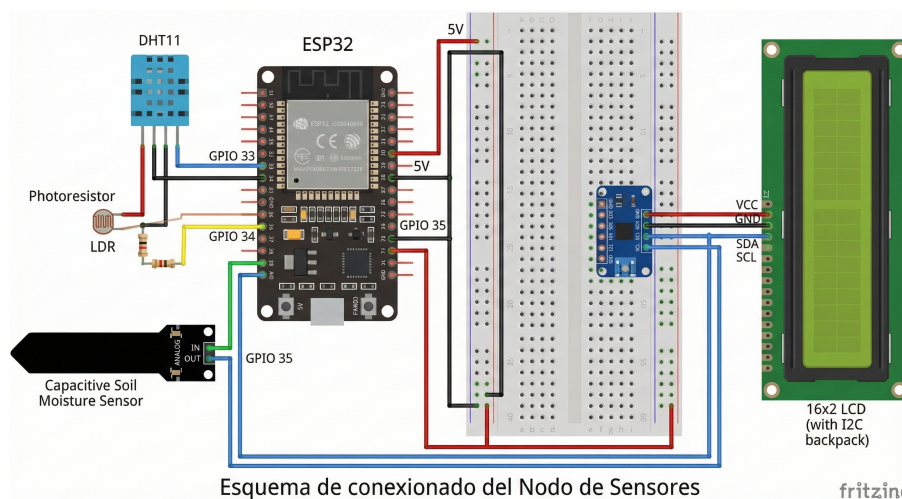


Figura 2: Esquema de conexionado del Nodo de Sensores.

2.2.1. Componentes y Conexión

Los actuadores están conectados a los siguientes pines de salida digital y PWM:

- **Bomba de Agua (Riego):** Controlada mediante el pin **GPIO 33**. Este pin activa un relé (o transistor de potencia) que cierra el circuito de la bomba de agua durante el tiempo de riego establecido (por defecto 30 segundos).
- **Servomotor (Ventana):** Conectado al pin **GPIO 23**. Se utiliza la librería **ESP32Servo** para controlar la apertura (180 grados) y cierre (0 grados) de la compuerta de ventilación.
- **Ventilador:** Controlado mediante el pin **GPIO 21**. Este actuador funciona en conjunto con el servomotor para forzar la circulación de aire cuando se activa el modo de ventilación (duración por defecto de 3 minutos).

2.3. Nodo de Cámara (Vigilancia)

Para la vigilancia visual se ha utilizado un módulo ESP32-CAM (modelo AI-Thinker). Este dispositivo es compacto e integra tanto la conectividad WiFi como el conector para la cámara OV2640.

2.3.1. Configuración Hardware

El conexionado de este nodo es mínimo, ya que la cámara se conecta directamente al zócalo FPC de la placa. Sin embargo, es relevante destacar el uso del LED flash integrado:

- **LED Flash:** Conectado internamente al **GPIO 4**. Se utiliza para iluminar la escena momentos antes de capturar la foto, asegurando visibilidad incluso en condiciones de baja luminosidad.

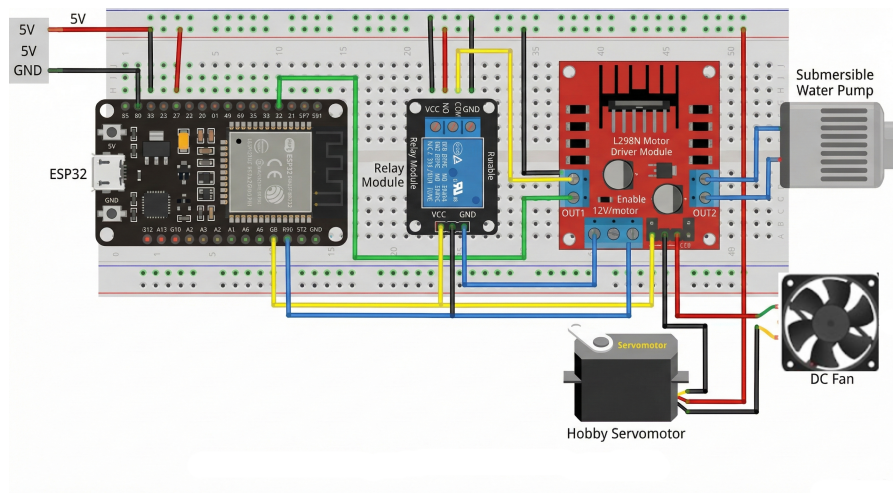


Figura 3: Esquema de conexionado del Nodo de Actuadores.

La alimentación de este módulo es crítica debido a los picos de corriente que se producen al utilizar el WiFi y la cámara simultáneamente. Se ha desactivado el detector de *Brownout* por software para evitar reinicios inestables, tal como se recomienda en aplicaciones alimentadas por batería o fuentes USB limitadas.

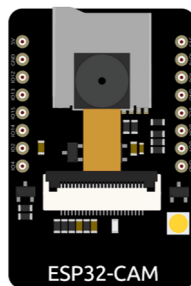


Figura 4: Módulo ESP32-CAM.

3. Diseño Software

El diseño software del sistema se ha estructurado en dos niveles: el *Firmware* embarcado en los microcontroladores ESP32 y la *Aplicación de Control* desplegada en el servidor central (Node-RED). A continuación se describe la lógica implementada en cada componente.

3.1. Firmware de los Dispositivos IoT

La programación de los nodos se ha realizado utilizando el IDE de Arduino con el soporte para las tarjetas ESP32. Se han implementado estrategias de programación diferenciadas según la función de cada nodo, destacando el uso del sistema operativo en tiempo real FreeRTOS para la gestión de tareas concurrentes.

3.1.1. Nodo de Sensores: Multitarea con FreeRTOS

Para el nodo de lectura de sensores, se ha diseñado una arquitectura basada en FreeRTOS para gestionar eficientemente las comunicaciones y la adquisición de datos de forma asíncrona. El firmware se divide en dos tareas principales (*Tasks*) sincronizadas mediante un semáforo binario:

1. **Tarea de Servicio MQTT (`taskMQTTService`):** Esta tarea tiene la prioridad más alta. Se encarga de inicializar la conexión WiFi y mantener la conexión con el Broker MQTT (`iot.ac.uma.es`). Una vez que la conexión es estable, libera un semáforo (`semMqttReady`) para indicar al resto del sistema que el canal de comunicación está listo.
2. **Tarea de Publicación (`taskPublisher`):** Esta tarea espera en estado de bloqueo (*Blocked*) hasta recibir la señal del semáforo `semMqttReady`. Una vez desbloqueada, procede a leer los sensores (DHT11, LDR, humedad de suelo), actualiza la pantalla LCD y formatea los datos en una cadena JSON. Finalmente, publica el mensaje en el topic `II11/sensores` y entra en modo de suspensión profunda (*Deep Sleep*) durante 5 minutos para ahorrar energía.

El uso de *Deep Sleep* permite que el dispositivo opere con un consumo energético mínimo entre lecturas, despertándose únicamente para tomar medidas y transmitir.

3.1.2. Nodo de Actuadores: Suscripción y Reacción

El nodo de actuadores implementa una lógica reactiva basada en suscripciones MQTT. Al iniciarse, se conecta al broker y se suscribe a los topics de control:

- `II11/actuadores/regar`
- `II11/actuadores/ventilar`

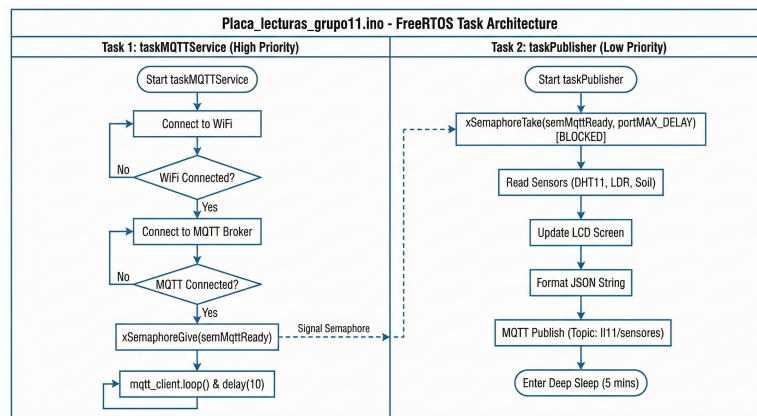


Figura 5: Diagrama de flujo de las tareas FreeRTOS en el nodo de sensores.

El bucle principal (`loop`) gestiona la recepción de mensajes. Al recibir un comando con el payload "1", se ejecuta la función correspondiente:

- **Función `regar()`:** Activa el pin digital de la bomba durante 30 segundos.
- **Función `ventilar()`:** Mueve el servomotor a 180 grados (apertura de ventana) y activa el ventilador. Tras 3 minutos, cierra la ventana (servo a 0 grados) y apaga el ventilador.

3.1.3. Nodo de Cámara: Captura Híbrida (MQTT/Temporizador) y Envío HTTP

El firmware del nodo cámara opera en modo continuo utilizando el sistema operativo en tiempo real **FreeRTOS**. Esto permite mantener la conexión WiFi y MQTT activas permanentemente para recibir órdenes externas, además de mantener un temporizador interno.

La arquitectura se basa en un modelo *Productor-Consumidor* gestionado mediante un semáforo binario (`semPhotoTrigger`), con el siguiente flujo de funcionamiento:

- **Inicialización:** Se configura el periférico de la cámara (resolución QVGA), se conecta al WiFi y se inician las tareas del sistema.
- **Disparadores (Productores):** El sistema espera eventos desde dos fuentes distintas para activar la captura:
 1. **Orden Remota MQTT:** El dispositivo se suscribe al topic `II11/camera/orden`. Al recibir cualquier mensaje en este canal, se libera el semáforo de captura.
 2. **Temporizador Automático:** Un *Software Timer* de FreeRTOS está configurado para dispararse cada hora (3600000 ms), liberando el semáforo automáticamente.
- **Procesamiento (Consumidor):** Una tarea dedicada (`taskPhotoConsumer`), que permanece bloqueada esperando el semáforo, realiza la secuencia de acción al activarse:
 1. Encendido del Flash (LED integrado) y captura de la imagen.
 2. Codificación de la imagen en formato Base64 y encapsulamiento en JSON.

3. Envío de la carga útil mediante una petición **HTTP POST** segura (HTTPS) al endpoint `/camera`.
4. Liberación de memoria y retorno al estado de espera (bloqueo) hasta el siguiente evento.

Esta configuración garantiza que el sistema sea reactivo a órdenes manuales inmediatas sin perder la funcionalidad de registro periódico.

3.2. Aplicación de Control y Supervisión (Node-RED)

El cerebro del sistema reside en un flujo de Node-RED que integra la recepción de datos, el almacenamiento, la visualización y la interacción con el usuario.

3.2.1. Gestión de Datos y Base de Datos

El sistema recibe los mensajes JSON del topic `II11/sensores`. Estos datos se procesan para añadirles una marca de tiempo (*timestamp*) legible y se almacenan en una base de datos MongoDB (colección `datos_json`). Esto permite generar históricos y gráficas de evolución. Las imágenes recibidas vía HTTP también se almacenan en una colección separada (`Fotos`).

3.2.2. Lógica de Automatización

Node-RED implementa la inteligencia del invernadero mediante nodos de función (*JavaScript*) que evalúan las condiciones ambientales en tiempo real:

- **Riego Automático:** Si la humedad del suelo cae por debajo del 55 %, se publica automáticamente una orden de riego en el topic MQTT de actuadores.
- **Ventilación Automática:** Si la humedad del aire supera el 73 % y hay suficiente luminosidad (indicativo de día), se activa el sistema de ventilación.

3.2.3. Integración con Servicios Externos (AEMET)

El sistema consulta periódicamente la API XML de la AEMET para obtener la predicción meteorológica de la zona. Esta información se procesa y se pone a disposición del usuario a través del bot de Telegram, permitiendo conocer si se esperan lluvias o cambios de temperatura que afecten al cultivo.

3.3. Interfaces de Usuario

Se han desarrollado dos interfaces principales para interactuar con el sistema, cumpliendo con los requisitos de amigabilidad y accesibilidad.

3.3.1. Dashboard Web

Se ha diseñado un panel de control gráfico (Dashboard) organizado en pestañas:

- **Menú Principal:** Navegación entre las distintas secciones.
- **Información Sensores:** Muestra indicadores tipo *Gauge* (agujas) para los valores actuales y gráficas lineales para la evolución histórica de temperatura, humedad y luminosidad.
- **Actuadores y Cámara:** Botones para forzar el riego, la ventilación o la toma de foto manualmente. Además de un visor para reproducir un *Time-Lapse* de las fotos almacenadas del invernadero.

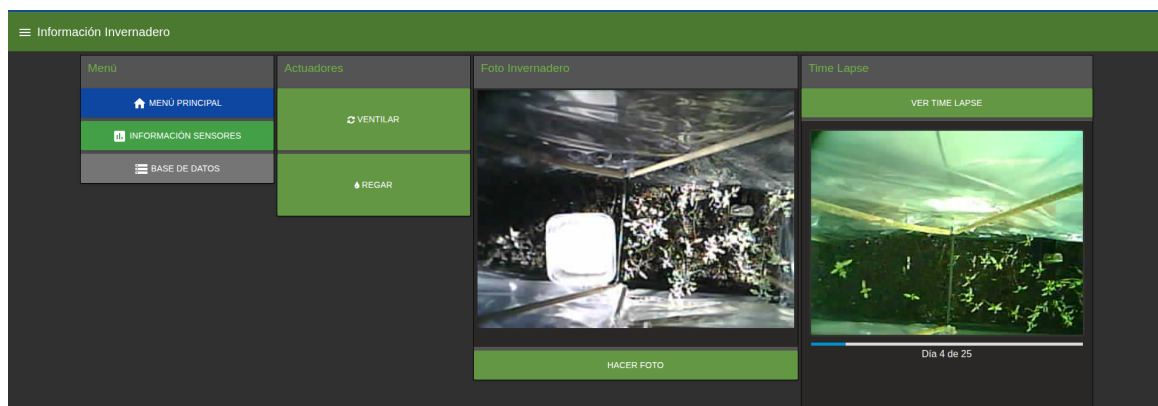


Figura 6: Captura de pantalla de la sección actuadores.

- **Base de Datos:** Tablas con los registros crudos y opciones para filtrar por fecha.

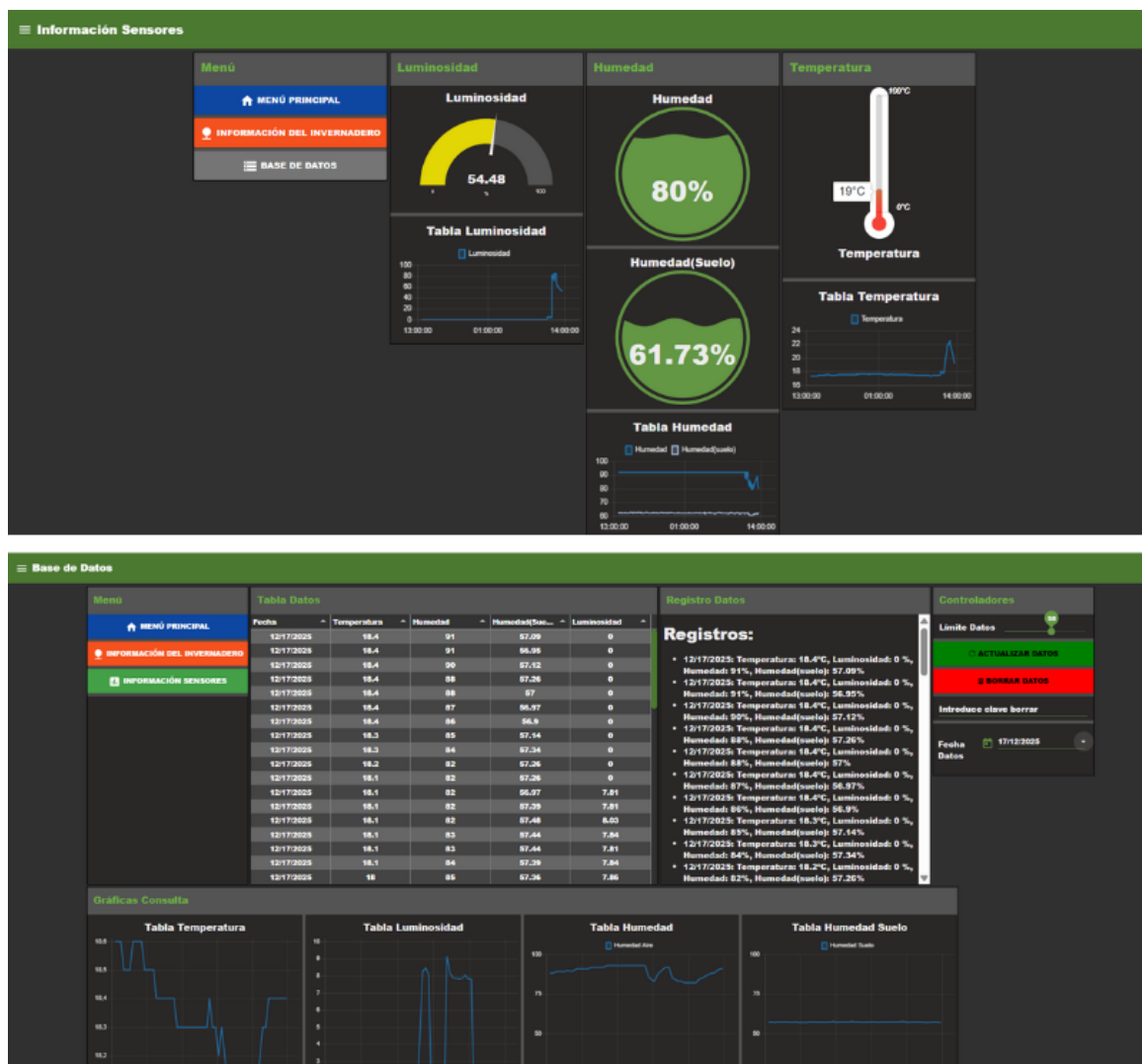


Figura 7: Captura de pantalla del Dashboard de control del invernadero.

3.3.2. Bot de Telegram

Se ha implementado un Bot conversacional (@Invernadero_Grupo11_Bot) que permite el control remoto desde cualquier lugar. El bot responde a los siguientes comandos:

- /start: Mensaje de bienvenida y lista de comandos.
- /datos: Devuelve un resumen instantáneo de los sensores (Temperatura, Humedad, Suelo, Luz).
- /tiempo: Muestra la predicción meteorológica de la AEMET.
- /regar y /ventilar: Solicitan la activación de los actuadores. Incluyen una lógica de seguridad: si los sensores indican que no es necesario (ej. el suelo ya está húmedo), el Bot pide confirmación al usuario mediante botones interactivos (*Inline Keyboards*) antes de ejecutar la acción.

4. Técnicas de Robustez y Eficiencia

Para asegurar el funcionamiento continuo del sistema ante posibles fallos de red o alimentación, se han implementado las siguientes estrategias software:

4.1. Gestión de Energía y Estabilidad

- **Prevención de Brownout:** En el nodo de la cámara, debido al alto consumo simultáneo del WiFi y el sensor de imagen, se ha desactivado por software el detector de caída de tensión (*Brownout Detector*). Esto evita reinicios constantes cuando la batería o la fuente USB tienen caídas momentáneas de voltaje.
- **Modo Deep Sleep:** El nodo de sensores entra en modo de suspensión profunda tras realizar su tarea. Esto no solo ahorra energía, sino que actúa como un reinicio limpio” periódico, evitando el desbordamiento de memoria (*heap fragmentation*) a largo plazo.
- **Interruptores analógicos:** La pantalla que muestra las medidas de los sensores se puede desconectar por medio de un interruptor. Esto es muy interesante puesto que la pantalla es el componente más costoso energéticamente hablando.

4.2. Tolerancia a Fallos de Comunicación

- **Reconexión Automática:** Los nodos implementan bucles de control que intentan restablecer la conexión WiFi y MQTT de forma indefinida si esta se pierde, impidiendo que el dispositivo quede en un estado irrecuperable.
- **Validación de Acciones Críticas:** En el control vía Telegram, se ha implementado una lógica de ”doble confirmación”. Si un usuario intenta regar cuando la humedad del suelo ya es alta, o ventilar cuando la humedad ambiente es baja, el sistema no ejecuta la orden directamente, sino que solicita confirmación explícita, protegiendo el cultivo de errores humanos.

4.3. Multitarea (FreeRTOS)

El uso de tareas independientes en el nodo de sensores y en el de la cámara permite que la pérdida temporal de conexión MQTT no bloquee la lectura de los sensores ni la actualización de la pantalla LCD local, desacoplando la lógica de adquisición de la lógica de comunicación.

5. Librerías Utilizadas

El desarrollo del firmware ha sido posible gracias al uso de las siguientes librerías estándar y de terceros:

■ Comunicación y Red:

- `WiFi.h`: Gestión de la conexión inalámbrica ESP32.
- `PubSubClient.h`: Cliente MQTT ligero para la comunicación con el broker.
- `HTTPClient.h` y `NetworkClientSecure.h`: Para el envío de imágenes vía POST HTTPS.

■ Hardware y Sensores:

- `DHTesp.h`: Lectura optimizada para sensores de temperatura y humedad DHT11/DHT22 en ESP32.
- `ESP32Servo.h`: Control preciso de servomotores utilizando los timers hardware del ESP32.
- `LiquidCrystal_I2C.h`: Control de pantallas LCD mediante bus I2C.
- `esp_camera.h`: Driver oficial para el manejo del sensor de imagen OV2640.

■ Utilidades:

- `base64.h`: Codificación de datos binarios (imágenes) a texto para su transmisión segura en JSON.

6. Resultados y Conclusiones

El proyecto ha resultado en la implementación exitosa de un invernadero automatizado funcional. Las pruebas realizadas demuestran la capacidad del sistema para mantener las condiciones ambientales dentro de los rangos establecidos de forma autónoma.

6.1. Logros Alcanzados

- Se ha logrado una integración estable entre dispositivos heterogéneos (ESP32 y ESP32-CAM) utilizando protocolos estándar (MQTT y HTTP).
- El Dashboard de Node-RED proporciona una visualización clara y en tiempo real, permitiendo el análisis histórico de datos gracias a la persistencia en MongoDB.
- El sistema de notificaciones y control vía Telegram ha demostrado ser una herramienta eficaz para la gestión remota, ofreciendo tiempos de respuesta inmediatos.
- La implementación de FreeRTOS ha mejorado significativamente la eficiencia del nodo de sensores en comparación con un enfoque secuencial tradicional (*super-loop*).

6.2. Conclusión

El sistema cumple con todos los requisitos funcionales y no funcionales planteados en la especificación del proyecto. La arquitectura modular adoptada facilita futuras ampliaciones, como la inclusión de más sensores o la integración de actuadores de calefacción, sin necesidad de rediseñar la red de comunicaciones existente.

7. Lista de Ficheros Entregados

Se adjuntan a esta memoria los siguientes archivos correspondientes al código fuente y configuración del sistema:

1. Firmware:

- `Placa_lecturas_grupo11.ino`: Código fuente del nodo de sensores (FreeRTOS).
- `Placa_actuadores_grupo11.ino`: Código fuente del nodo de actuadores.
- `camara_http_post_definitivo.ino`: Código fuente del nodo de cámara.

2. Software de Control:

- `flows.json`: Flujo completo de Node-RED (Dashboard, Telegram, Lógica y Base de Datos).

3. Documentación:

- `Memoria_Proyecto_Grupo11.pdf`: Este documento.
- Manual de Usuario.